

MODULE 4 – FORMULAIRES & NAVIGATION

Formation Flutter Professionnelle

SUPPORT DE COURS – Théorie

1. Saisies utilisateur : TextField et TextFormField

TextField (champ de texte basique)

```
TextField(  
  decoration: InputDecoration(  
    labelText: 'Nom d'utilisateur',  
    hintText: 'Entrez votre nom',  
    prefixIcon: Icon(Icons.person),  
    border: OutlineInputBorder(),  
  ),  
  onChanged: (value) {  
    print('Saisie : $value');  
  },  
)
```

Propriétés clés :

- **controller** : contrôle programmatique du texte (lecture/écriture)
- **onChanged** : callback à chaque frappe
- **onSubmitted** : callback quand l'utilisateur appuie sur "Entrée"
- **keyboardType** : type de clavier (text, number, email, phone, datetime...)
- **obscureText** : masque le texte (pour les mots de passe)
- **maxLines** : nombre de lignes (1 par défaut, null pour multi-ligne)
- **autofocus** : focus automatique au montage du widget **TextFormField** (champ de formulaire avec validation)

```
TextFormField(  
  decoration: const InputDecoration(  
    labelText: 'Email',  
    hintText: 'exemple@email.com',  
    prefixIcon: Icon(Icons.email),  
    border: OutlineInputBorder(),  
  ),  
  keyboardType: TextInputType.emailAddress,  
  validator: (value) {  
    if (value == null || value.isEmpty) {  
      return 'Ce champ est obligatoire';  
    }  
    if (!value.contains('@')) {  
      return 'Veuillez entrer un email valide';  
    }  
  },  
)
```

```

    }
    return null; // Pas d'erreur
  },
)

```

Différence fondamentale :

	TextField	TextFormField
Validation	Non intégrée	validator intégré
Formulaire	Utilisé seul	Utilisé dans un Form
Réinitialisation	Manuelle	Via Form avec GlobalKey
Cas d'usage	Recherche, chat	Inscription, login, commande

InputDecoration – L’art du champ de saisie

```

InputDecoration(
  // Label flottant (monte quand le champ est focus ou rempli)
  labelText: 'Mot de passe',
  // Texte d'aide affiché en permanence sous le champ
  helperText: 'Minimum 8 caractères',
  // Texte indicatif grisé dans le champ vide
  hintText: '.....',
  // Icône à gauche du champ
  prefixIcon: Icon(Icons.lock),
  // Icône à droite (ex: œil pour afficher/masquer le mot de passe)
  suffixIcon: IconButton(
    icon: Icon(_estVisible ? Icons.visibility : Icons.visibility_off),
    onPressed: () => setState(() => _estVisible = !_estVisible),
  ),
  // Bordure par défaut
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
  ),
  // Bordure quand le champ est focus
  focusedBorder: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
    borderSide: BorderSide(color: Colors.deepPurple, width: 2),
  ),
  // Bordure en cas d'erreur de validation
  errorBorder: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
    borderSide: BorderSide(color: Colors.red, width: 2),
  ),
  // Remplissage du champ
  filled: true,
  fillColor: Colors.grey.shade50,
)

```

GlobalKey – Le cerveau du formulaire

```
class _PageInscriptionState extends State<PageInscription> {

  // GlobalKey permet d'accéder à l'état du widget Form depuis n'importe où
  // dans la classe. C'est indispensable pour valider, sauvegarder ou
  // réinitialiser le formulaire programmatically.
  final _formKey = GlobalKey<FormState>();

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey, // Association de la clé au formulaire
      child: Column(
        children: [
          TextFormField(
            validator: (value) => value!.isEmpty ? 'Champ requis' : null,
          ),
          ElevatedButton(
            onPressed: () {
              // Valide TOUS les TextFormField du formulaire
              // Retourne true si tous les validators retournent null
              if (_formKey.currentState!.validate()) {
                // Sauvegarde les valeurs de tous les champs
                _formKey.currentState!.save();
                // Traitement des données...
              }
            },
            child: const Text('Valider'),
          ),
        ],
      ),
    );
  }
}
```

Méthodes de FormState :

- validate() : exécute tous les validators, affiche les erreurs, retourne bool
- save() : appelle onSave de chaque TextFormField
- reset() : efface tous les champs et les erreurs

2. Composants de formulaire avancés

DropDownButton (liste déroulante)

```
String? _categorieSelectionnee;
```

```

DropdownButtonFormField<String>({
  value: _categorieSelectionnee,
  decoration: const InputDecoration(
    labelText: 'Catégorie',
    border: OutlineInputBorder(),
  ),
  items: const [
    DropdownMenuItem(value: 'design', child: Text('Design')),
    DropdownMenuItem(value: 'backend', child: Text('Backend')),
    DropdownMenuItem(value: 'qualite', child: Text('Qualité')),
  ],
  onChanged: (value) {
    setState(() => _categorieSelectionnee = value);
  },
  validator: (value) => value == null ? 'Sélectionnez une catégorie' : null,
})

```

DropdownButtonFormField vs DropdownButton :

- DropdownButtonFormField s'intègre dans un Form, a un validator, un style cohérent avec TextFormField.
- DropdownButton est autonome, plus léger, mais sans validation intégrée.

Checkbox, Switch, Radio

// Checkbox (case à cocher)

```

CheckboxListTile(
  title: const Text('J'accepte les conditions d'utilisation'),
  value: _conditionsAcceptees,
  onChanged: (value) {
    setState(() => _conditionsAcceptees = value ?? false);
  },
  controlAffinity: ListTileControlAffinity.leading,
)

```

// Switch (interrupteur on/off)

```

SwitchListTile(
  title: const Text('Activer les notifications'),
  subtitle: const Text('Recevoir des alertes par email'),
  value: _notificationsActivees,
  onChanged: (value) {
    setState(() => _notificationsActivees = value);
  },
)

```

// Radio (choix unique dans un groupe)

```

RadioListTile<String>({
  title: const Text('Light'),

```

```

value: 'light',
groupValue: _theme,
onChanged: (value) => setState(() => _theme = value!),
)

```

DatePicker et TimePicker

```

// Sélecteur de date
Future<void> _selectionnerDate() async {
  final DateTime? date = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(2020),
    lastDate: DateTime(2030),
  );
  if (date != null) {
    setState(() => _dateSelectionnee = date);
  }
}

// Sélecteur d'heure
Future<void> _selectionnerHeure() async {
  final TimeOfDay? heure = await showTimePicker(
    context: context,
    initialTime: TimeOfDay.now(),
  );
  if (heure != null) {
    setState(() => _heureSelectionnee = heure);
  }
}

```

Bonnes pratiques formulaires :

1. **Toujours utiliser TextFormField dans un Form** pour bénéficier de la validation centralisée.
2. **Toujours fournir un labelText** pour l'accessibilité (lecteurs d'écran).
3. **Valider côté client ET côté serveur** – la validation Flutter n'est pas une sécurité.
4. **Utiliser autovalidateMode: AutovalidateMode.onUserInteraction** pour valider après la première interaction, pas dès le montage (évite d'afficher des erreurs avant que l'utilisateur n'ait commencé à taper).
5. **Gérer le clavier** : `TextInputAction.next` pour passer au champ suivant, `TextInputAction.done` pour le dernier champ.

3. Navigation classique avec Navigator

Système de navigation impératif (Navigator 1.0) Flutter utilise une pile (stack) d'écrans. Chaque nouvel écran est empilé au-dessus du précédent.

```

// Navigation simple (push)

```

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const PageDetail(),
  ),
);

// Navigation avec retour de données
final resultat = await Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const PageFormulaire()),
);
// resultat contient la valeur passée à Navigator.pop()

// Retour en arrière (pop)
Navigator.pop(context);

// Retour avec données
Navigator.pop(context, 'Données retournées');

// Remplacer l'écran actuel (pas de retour possible)
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const PageTableauBord()),
);

// Vider la pile et aller à un écran (déconnexion → login)
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (context) => const PageLogin()),
  (route) => false, // Supprime TOUTES les routes précédentes
);
```

MaterialPageRoute vs CupertinoPageRoute :

- MaterialPageRoute : transition slide + fade (Android/Material)
- CupertinoPageRoute : transition slide horizontal (iOS style)

Routes nommées

Définies dans MaterialApp, utiles pour les petites applications.

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const PageAccueil(),
    '/login': (context) => const PageLogin(),
  },
);
```

```

    '/detail': (context) => const PageDetail(),
  },
);

// Navigation
Navigator.pushNamed(context, '/login');

// Avec arguments
Navigator.pushNamed(
  context,
  '/detail',
  arguments: {'id': '123', 'titre': 'Tâche 1'},
);

// Récupération des arguments
final args = ModalRoute.of(context)!.settings.arguments as Map<String, dynamic>;

```

Limites de Navigator 1.0 :

- Pas de navigation profonde (deep linking) native
- Difficile de gérer des URL web
- Pas de guards d'authentification élégants
- Gestion des paramètres peu typée

4. go_router – Navigation déclarative moderne

go_router est le package de navigation recommandé par l'équipe Flutter. Il résout toutes les limites de Navigator 1.0.

Installation

```

dependencies:
  go_router: ^14.0.0

```

Configuration de base

```

import 'package:go_router/go_router.dart';

// Déclaration des routes en dehors de tout widget
// C'est une configuration pure (pas de BuildContext nécessaire)
final GoRouter _router = GoRouter(
  // Route initiale au lancement de l'app
  initialLocation: '/',

  // Liste des routes déclaratives
  routes: [
    GoRoute(

```

```
    path: '/',
    builder: (context, state) => const PageAccueil(),
  ),
  GoRoute(
    path: '/login',
    builder: (context, state) => const PageLogin(),
  ),
  GoRoute(
    path: '/detail/:id', // Paramètre dynamique dans l'URL
    builder: (context, state) {
      // Récupération du paramètre typé
      final id = state.pathParameters['id']!;
      return PageDetail(id: id);
    },
  ),
],
);
```

Intégration dans MaterialApp

```
class MonApplication extends StatelessWidget {
  const MonApplication({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      // MaterialApp.router remplace MaterialApp quand on utilise go_router
      routerConfig: _router,
      title: 'Mon Application',
      theme: ThemeData(useMaterial3: true),
    );
  }
}
```

Navigation avec go_router

```
import 'package:go_router/go_router.dart';

// Navigation simple (push)
context.go('/login'); // Remplace l'écran actuel
context.push('/detail/123'); // Empile un nouvel écran (retour possible)

// Navigation avec paramètres
context.go('/detail/123');

// Navigation avec extra (données complexes)
context.push('/detail/123', extra: monObjetTache);
```



```
// Retour en arrière
context.pop();

// Retour avec résultat
context.pop('Résultat');

// Aller à la racine (vider la pile)
context.go('/');
```

Différence go vs push :

Méthode	Comportement	Quand l'utiliser ?
go()	Remplace la route actuelle (pas d'historique)	Login → Dashboard, changement de section principale
push()	Empile la nouvelle route (retour possible)	Liste → Détail, formulaire en overlay

Redirections et guards d'authentification

```
final GoRouter _router = GoRouter(
  initialLocation: '/',

  // redirect est appelé à CHAQUE changement de route
  // C'est le système de "route guards" de go_router
  redirect: (context, state) {
    final estAuthentifie = AuthService.of(context).estConnecte;
    final vaVersLogin = state.matchedLocation == '/login';

    // Si non authentifié et pas déjà sur login → redirige vers login
    if (!estAuthentifie && !vaVersLogin) {
      return '/login';
    }

    // Si authentifié et sur login → redirige vers accueil
    if (estAuthentifie && vaVersLogin) {
      return '/';
    }

    // Pas de redirection (null = laisser passer)
    return null;
  },

  routes: [
```

```
GoRoute(path: '/', builder: (_, __) => const PageAccueil()),
GoRoute(path: '/login', builder: (_, __) => const PageLogin()),
GoRoute(
  path: '/profil',
  builder: (_, __) => const PageProfil(),
  // Routes enfants (nested navigation)
  routes: [
    GoRoute(
      path: 'parametres', // URL complète : /profil/parametres
      builder: (_, __) => const PageParametres(),
    ),
  ],
),
],
);
```

Routes enfants et ShellRoute (navigation avancée)

```
final GoRouter _router = GoRouter(
  routes: [
    // ShellRoute enveloppe plusieurs routes dans une structure commune
    // (ex: BottomNavigationBar persistant)
    ShellRoute(
      builder: (context, state, child) {
        return ScaffoldWithNavBar(child: child); // BottomNav persistant
      },
      routes: [
        GoRoute(path: '/', builder: (_, __) => const PageAccueil()),
        GoRoute(path: '/taches', builder: (_, __) => const PageTaches()),
        GoRoute(path: '/profil', builder: (_, __) => const PageProfil()),
      ],
    ),
  ],
);
```

Pourquoi go_router est supérieur à Navigator 1.0 :

-  **Deep linking natif** /detail/123 fonctionne directement
-  **Web-friendly**: URLs réelles, historique navigateur
-  **Typage fort** paramètres extra typés, pas de dynamic
-  **Redirections déclaratives** guards centralisés, pas de spaghetti
-  **Nested routing** navigation dans des onglets sans perdre l'état
-  **Animations personnalisées** transitions par route

ÉNONCÉS DES TRAVAUX PRATIQUES

TP1 – Écrans de Connexion et d'Inscription (1h 15min)

Contexte Vous créez les écrans d'authentification de votre application fil rouge. Ces écrans seront réutilisés dans le Module 7 (JWT) et le Module 8 (Firebase Auth).

Objectifs pédagogiques

- Maîtriser TextFormField avec validation complète
- Utiliser GlobalKey<FormState> pour la validation centralisée
- Gérer l'affichage/masquage du mot de passe
- Créer une UI d'authentification professionnelle

Maquette – Page de connexion

Consignes Étape 1 – Créer la page de connexion Dans lib/screens/login_page.dart, créez un StatefulWidgetPageLogin avec :

- Un Scaffold sans AppBar (design épuré)
- Un SafeArea avec SingleChildScrollView pour éviter les débordements
- Un Padding de 24dp de tous les côtés
- Une Column avec crossAxisAlignment: CrossAxisAlignment.stretch

Contenu de la Column :

1. En-tête :

- Icon(Icons.account_circle, size: 80, color: primary)
- SizedBox(height: 24)
- Text('Bienvenue !', style: headlineLarge, bold)
- Text('Connectez-vous pour continuer', style: bodyMedium, grey)
- SizedBox(height: 32)

2. Formulaire (avec GlobalKey<FormState>) :

- TextFormField Email :
 - labelText: 'Email'
 - prefixIcon: Icons.email_outlined
 - keyboardType: TextInputType.emailAddress
 - textInputAction: TextInputAction.next
 - Validator : non vide + format email (regex simple : `^[^@]+@[^@]+\.[^@]+`)
- SizedBox(height: 16)
- TextFormField Mot de passe :
 - labelText: 'Mot de passe'
 - prefixIcon: Icons.lock_outline
 - obscureText: !_estVisible (géré par un booléen d'état)
 - suffixIcon: IconButton avec œil pour toggle visibility
 - textInputAction: TextInputAction.done
 - Validator : non vide + minimum 6 caractères
- SizedBox(height: 8)
- Row avec Checkbox "Se souvenir de moi" aligné à gauche
- SizedBox(height: 24)

- ElevatedButton “SE CONNECTER” :
 - onPressed : valide le formulaire avec `_formKey.currentState!.validate()`
 - Si valide : affiche un SnackBar “Connexion réussie (simulation)”
 - Style : fond primary, texte blanc, padding vertical 16, borderRadius 12
- SizedBox(height: 16)
- TextButton “Pas de compte ? S’inscrire” qui navigue vers `/register`

Étape 2 – Créer la page d’inscription Dans `lib/screens/register_page.dart`, même structure avec des champs supplémentaires :

- **Nom complet** (validator : minimum 2 caractères)
- **Email** (même validation que login)
- **Mot de passe** (validator : minimum 8 caractères, 1 majuscule, 1 chiffre)
- **Confirmation mot de passe** (validator : doit être identique au mot de passe)
- **Dropdown** “Rôle” : Développeur, Designer, Chef de projet
- **Checkbox** “J’accepte les conditions d’utilisation” (validator : doit être coché)
- **Bouton** “S’INSCRIRE”

Étape 3 – Gestion d’état locale Utilisez `setState` pour :

- Toggle visibilité mot de passe
- Toggle “Se souvenir de moi”
- Toggle “Conditions acceptées”
- Stocker les valeurs des champs (via `TextEditingController` ou `onSaved`) **Livrable attendu** : Code source des deux pages + captures d’écran.

TP2 – Configuration GoRouter et SplashScreen (45 min)

Contexte Vous remplacez la navigation manuelle par `go_router` et vous ajoutez un écran de lancement qui redirige vers le bon écran selon l’état de connexion.

Objectifs pédagogiques

- Configurer `go_router` avec routes déclaratives
- Implémenter une redirection conditionnelle
- Créer un `SplashScreen` avec animation

Consignes Étape 1 – Ajouter la dépendance

```
flutter pub add go_router
```

Étape 2 – Créer le fichier de routing Dans `lib/core/router.dart`, créez le router :

```
import 'package:go_router/go_router.dart';
import 'package:flutter/material.dart';
import '../screens/...';
```

```
final router = GoRouter(
  initialLocation: '/splash',
  routes: [
    GoRoute(
      path: '/splash',
      builder: (context, state) => const SplashScreen(),
    ),
    GoRoute(
      path: '/login',
      builder: (context, state) => const PageLogin(),
    ),
    GoRoute(
      path: '/register',
      builder: (context, state) => const PageRegister(),
    ),
    GoRoute(
      path: '/',
      builder: (context, state) => const PageAccueil(),
    ),
    GoRoute(
      path: '/detail/:id',
      builder: (context, state) {
        final id = state.pathParameters['id'];
        return PageDetail(tacheId: id);
      },
    ),
  ],
);
```

Étape 3 – Modifier main.dart Remplacez MaterialApp par MaterialApp.router :

```
class MonApplication extends StatelessWidget {
  const MonApplication({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      routerConfig: router,
      title: 'Gestion de Tâches',
      theme: AppTheme.themeClair,
      debugShowCheckedModeBanner: false,
    );
  }
}
```

Étape 4 – Créer le SplashScreen Dans lib/screens/splash_screen.dart :

```
class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});
```

```

@override
State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    // Attendre 2 secondes puis rediriger
    Future.delayed(const Duration(seconds: 2), () {
      // Vérifier si l'utilisateur est connecté (simulation)
      final estConnecte = false; // À remplacer par vraie logique plus tard

      if (estConnecte) {
        context.go('/');
      } else {
        context.go('/login');
      }
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.task_alt, size: 80, color: Theme.of(context).colorScheme.primary),
          const SizedBox(height: 24),
          const Text('Gestion de Tâches', style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
          const SizedBox(height: 48),
          const CircularProgressIndicator(),
        ],
      ),
    ),
  );
}
}

```

Étape 5 – Remplacer toutes les navigations Dans PageLogin, remplacez Navigator.push par context.push('/register'). Dans PageAccueil, utilisez context.push('/detail/123') pour le détail.

Livrable attendu : Code source du router + SplashScreen + pages modifiées.

FICHE RÉCAP STAGIAIRE – Formulaires & Navigation

Points clés à retenir

Table 4 – continued	
Concept	Syntaxe / Règle
TextFormField	Champ de formulaire avec validation intégrée
TextField	Champ de saisie simple (hors formulaire)
GlobalKey	Clé d'accès au formulaire pour valider/sauvegarder
validate()	Exécute tous les validateurs, retourne true si OK
save()	Appelle onSave de chaque champ
InputDecoration	Style du champ : label, hint, icônes, bordures
DropdownButtonFormField	Liste déroulante validable dans un Form
CheckboxListTile	Case à cocher avec label intégré
SwitchListTile	Interrupteur on/off avec label
showDatePicker	Sélecteur de date natif
Navigator.push	Empile un écran (retour possible)
Navigator.pushReplacement	Remplace l'écran actuel
context.go()	go_router : remplace la route
Concept	Syntaxe / Règle
context.push()	go_router : empile la route
context.pop()	Retour à l'écran précédent
pathParameters	Paramètres d'URL dans go_router

Structure type d'un formulaire Flutter

```
class MonFormulaire extends StatefulWidget {
  @override
  State<MonFormulaire> createState() => _MonFormulaireState();
}

class _MonFormulaireState extends State<MonFormulaire> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose(); // IMPORTANT : évite les fuites mémoire
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
```

```

return Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        controller: _emailController,
        decoration: const InputDecoration(labelText: 'Email'),
        validator: (value) {
          if (value == null || value.isEmpty) return 'Champ requis';
          if (!value.contains('@')) return 'Email invalide';
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Traiter les données
          }
        },
        child: const Text('Valider'),
      ),
    ],
  ),
);
}

```

Erreurs fréquentes à éviter

1. **Oublier GlobalKey<FormState>** → impossible de valider le formulaire
2. **Ne pas disposer les TextEditingController** → fuite mémoire
3. **Utiliser TextField au lieu de TextFormField dans un Form** → pas de validation
4. **Oublier return null dans le validator** → le champ est toujours invalide
5. **Ne pas gérer le clavier avec TextInputAction** → mauvaise UX (pas de “Suivant”)
6. **Utiliser Navigator et go_router mélangés** → comportements imprévisibles
7. **Oublier MaterialApp.router** → go_router ne fonctionne pas
8. **Ne pas tester la validation avec des données invalides** → bugs en production

Check mentale avant de coder un formulaire

- ☐ Ai-je besoin de validation ? → Form + TextFormField
- ☐ Mes champs ont-ils un label et un hint ? → Accessibilité
- ☐ Le clavier s'adapte-t-il au type de champ ? → keyboardType
- ☐ Puis-je passer au champ suivant avec "Suivant" ? → textInputAction
- ☐ Le mot de passe est-il masquable ? → suffixIcon + obscureText
- ☐ Ai-je un bouton de validation centralisé ? → _formKey.currentState!.validate()
- ☐ Mes controllers sont-ils disposés ? → dispose()

□ La navigation est-elle cohérente (go_router) ? → context.go() ou context.push()

QUIZ DE VALIDATION – Module 4

Durée : 10 minutes – 8 questions

Question 1 (QCM)

Quel widget permet de valider un champ de texte dans un formulaire ?

- A. TextField
- B. TextFormField
- C. InputField
- D. FormText

Question 2 (QCM)

Quelle méthode valide TOUS les champs d'un formulaire ?

- A. form.validateAll()
- B. formKey.currentState!.validate()
- C. form.check()
- D. form.verify()

Question 3 (Ouvverte)

Quelle est la différence entre context.go('/login') et context.push('/login') dans go_router ?

Réponse attendue :

- go() remplace la route actuelle (pas d'historique, pas de retour possible).
- push() empile la nouvelle route (l'utilisateur peut revenir en arrière avec pop()).

Question 4 (QCM)

Quel paramètre de TextFormField permet de masquer un mot de passe ?

- A. hiddenText: true
- B. obscureText: true
- C. maskText: true
- D. secureText: true

Question 5 (Ouvverte)

Écrivez un validator pour un champ "Nom" qui vérifie qu'il n'est pas vide et contient au moins 2 caractères.

Réponse attendue :

```
validator: (value) {  
  if (value == null || value.isEmpty) return 'Le nom est obligatoire';  
}
```

```
if (value.length < 2) return 'Le nom doit contenir au moins 2 caractères';  
return null;  
}
```

Question 6 (QCM)

Dans `go_router`, comment récupère-t-on un paramètre d'URL comme `/detail/:id` ?

- A. `state.arguments['id']`
- B. `state.pathParameters['id']`
- C. `state.params['id']`
- D. `state.query['id']`

Question 7 (QCM)

Pourquoi faut-il disposer un `TextEditingController` dans `dispose()` ?

- A. Pour effacer le texte
- B. Pour éviter les fuites de mémoire
- C. Pour fermer le clavier
- D. Pour valider le formulaire

Question 8 (QCM – difficulté +)

Quel widget permet d'afficher une liste déroulante VALIDABLE dans un Form ?

- A. `DropDownButton`
- B. `DropDownButtonFormField`
- C. `SelectFormField`
- D. `ListFormField`

Barème indicatif

- 8/8 : Prêt pour le Module 5 (Riverpod)
- 6-7/8 : Acquis solides, réviser `go_router` vs `Navigator`
- 4-5/8 : Refaire les TP, le Module 5 sera difficile
- < 4/8 : Revoir la théorie formulaires et navigation

ANNEXE – Ressources et liens

- **go_router documentation** : https://pub.dev/packages/go_router
- **Formulaires Flutter** : <https://docs.flutter.dev/cookbook/forms>
- **Validation** : <https://docs.flutter.dev/cookbook/forms/validation>
- **TextFormField** : <https://api.flutter.dev/flutter/material/TextFormField-class.html>
- **InputDecoration** : <https://api.flutter.dev/flutter/material/InputDecoration-class.html>
- **DatePicker** : <https://docs.flutter.dev/reference/flutter-cli>

